Cloud and Mobile Computing

Lab 1 Report

Exploring Cloud Virtualization and Data Center Architecture

Student Name: Bassel Adel

Group: SE3

Lab Title: Cloud Virtualization & Data Center Architecture

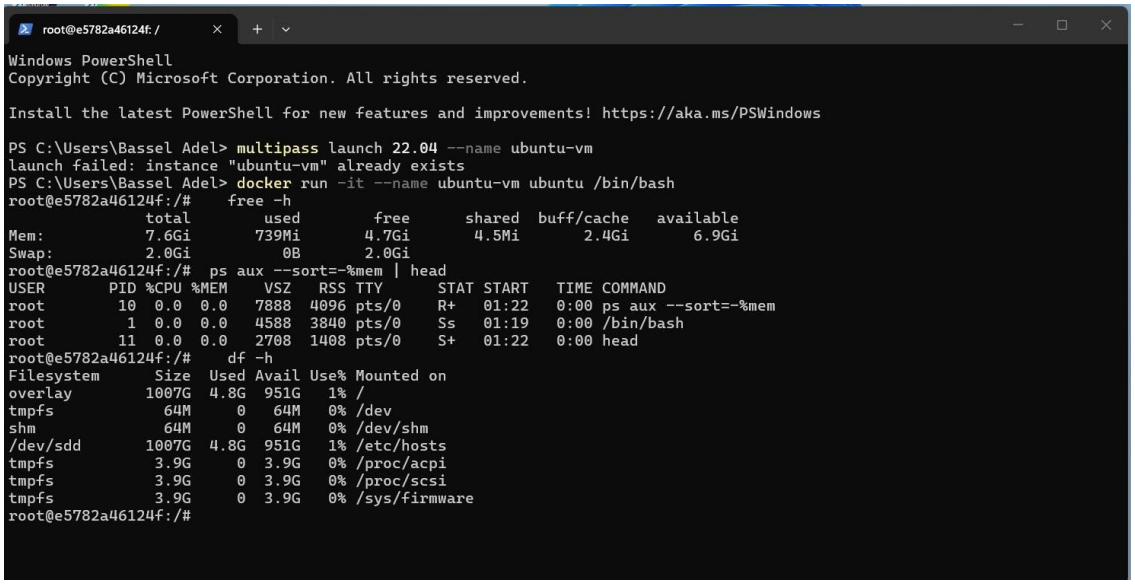
Learning Objectives

- Understand the difference between VMs and containers in practice.
- Deploy and compare instances on cloud and local environments.
- Visualize a warehouse-scale architecture and analyze tail latency patterns.
- Experiment with virtualization overhead and container density.

Part A: VMs vs Containers Comparison

In this part, two Docker containers were launched locally to simulate and compare VM-like and container environments. Resource usage was measured using standard Linux commands: free -h (memory), ps aux (processes), and df -h (disk).

Container 1 — Resource Usage



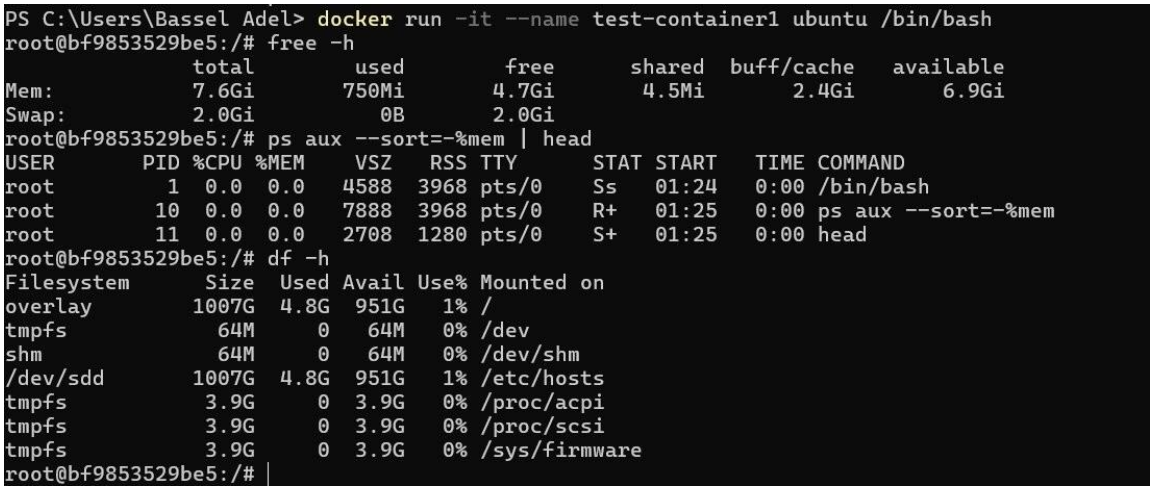
```
root@e5782a46124f: /
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Bassel Adel> multipass launch 22.04 --name ubuntu-vm
launch failed: instance "ubuntu-vm" already exists
PS C:\Users\Bassel Adel> docker run -it --name ubuntu-vm ubuntu /bin/bash
root@e5782a46124f:/#    free -h
               total        used        free      shared  buff/cache   available
Mem:           7.6Gi       739Mi       4.7Gi       4.5Mi       2.4Gi       6.9Gi
Swap:          2.0Gi          0B       2.0Gi
root@e5782a46124f:/#  ps aux --sort=-%mem | head
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root        10  0.0  0.0   7888  4096 pts/0    R+   01:22   0:00 ps aux --sort=-%mem
root         1  0.0  0.0   4588  3840 pts/0    Ss   01:19   0:00 /bin/bash
root        11  0.0  0.0   2708  1408 pts/0    S+   01:22   0:00 head
root@e5782a46124f:/#    df -h
Filesystem      Size  Used Avail Use% Mounted on
overlay        1007G  4.8G  951G   1% /
tmpfs            64M     0   64M   0% /dev
shm              64M     0   64M   0% /dev/shm
/dev/sdd       1007G  4.8G  951G   1% /etc/hosts
tmpfs           3.9G     0  3.9G   0% /proc/acpi
tmpfs           3.9G     0  3.9G   0% /proc/scsi
tmpfs           3.9G     0  3.9G   0% /sys/firmware
root@e5782a46124f:/#
```

Figure 1: Container 1 — free -h, ps aux, df -h output

Container 2 — Resource Usage



```
PS C:\Users\Bassel Adel> docker run -it --name test-container1 ubuntu /bin/bash
root@bf9853529be5:/# free -h
               total        used        free      shared  buff/cache   available
Mem:           7.6Gi       750Mi       4.7Gi       4.5Mi       2.4Gi       6.9Gi
Swap:          2.0Gi          0B       2.0Gi
root@bf9853529be5:/# ps aux --sort=-%mem | head
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.0   4588  3968 pts/0    Ss   01:24   0:00 /bin/bash
root        10  0.0  0.0   7888  3968 pts/0    R+   01:25   0:00 ps aux --sort=-%mem
root        11  0.0  0.0   2708  1280 pts/0    S+   01:25   0:00 head
root@bf9853529be5:/# df -h
Filesystem      Size  Used Avail Use% Mounted on
overlay        1007G  4.8G  951G   1% /
tmpfs            64M     0   64M   0% /dev
shm              64M     0   64M   0% /dev/shm
/dev/sdd       1007G  4.8G  951G   1% /etc/hosts
tmpfs           3.9G     0  3.9G   0% /proc/acpi
tmpfs           3.9G     0  3.9G   0% /proc/scsi
tmpfs           3.9G     0  3.9G   0% /sys/firmware
root@bf9853529be5:/#
```

Figure 2: Container 2 — free -h, ps aux, df -h output

Observations

Metric	Docker Container	Real VM (expected)
Memory Used	~739–750 MB	1–2+ GB
Running Processes	3 (bash, ps, head)	50–100+ (systemd, etc.)

Startup Time	Seconds	Minutes
Disk Size	1007 GB (shared host)	Fixed allocated disk

Part B: Cloud Infrastructure Exploration (AWS EC2)

A t3.micro EC2 instance was launched on AWS Free Tier running Amazon Linux. The Nitro Hypervisor was explored by running dmidecode and htop commands via EC2 Instance Connect in the browser.

EC2 Instance Launch

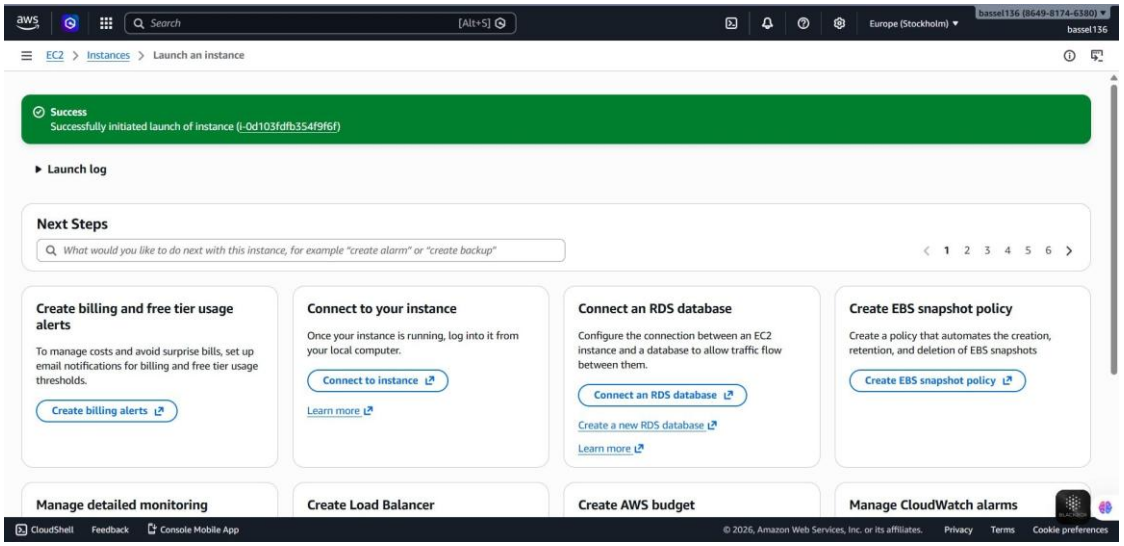


Figure 3: Successful EC2 instance launch on AWS (t3.micro, Europe/Stockholm region)

Nitro Hypervisor & System Info

Running `dmesg | grep -i nitro` returned an 'Operation not permitted' error, demonstrating that the Nitro hypervisor enforces strict security and blocks direct kernel buffer access. Running `sudo dmidecode` confirmed the instance is running on Amazon EC2 infrastructure:

```
Manufacturer: Amazon EC2    |    Product Name: t3.micro    |    Version: Not Specified
```

htop — Process Viewer on EC2

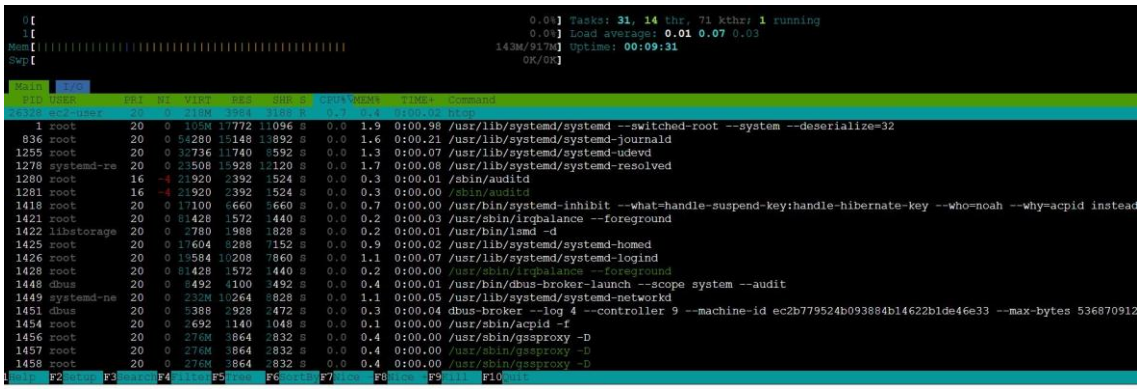


Figure 4: htop on EC2 showing 31 tasks, 14 threads — significantly more than a Docker container

Key Observation

The EC2 VM runs 31 background tasks including systemd, journald, auditd, networkd, and dbus — compared to only 3 processes in the Docker container. This confirms that VMs carry a much higher overhead due to running a full OS stack, while containers share the host kernel and are far more lightweight.

Part C: Tail Latency Mini-Simulation

A Flask web application was deployed locally with an artificial exponential delay (mean 100ms) to simulate real-world tail latency behavior. 100 sequential HTTP requests were made using a Python benchmarking script.

Flask App Code

```
from flask import Flask
import time, random
app = Flask(__name__)

@app.route('/')
def hello():
    delay = random.expovariate(1/0.1)
    time.sleep(delay)
    return f"Response after {delay:.2f} seconds"

app.run(host='0.0.0.0', port=5000)
```

Benchmark Results

Metric	Value
Total Requests	100
Minimum Latency	2014.08 ms
Maximum Latency	2387.74 ms
Average Latency	2126.29 ms
P90 (Tail Latency)	2217.14 ms
P99 (Tail Latency)	2320.09 ms

Latency Histogram

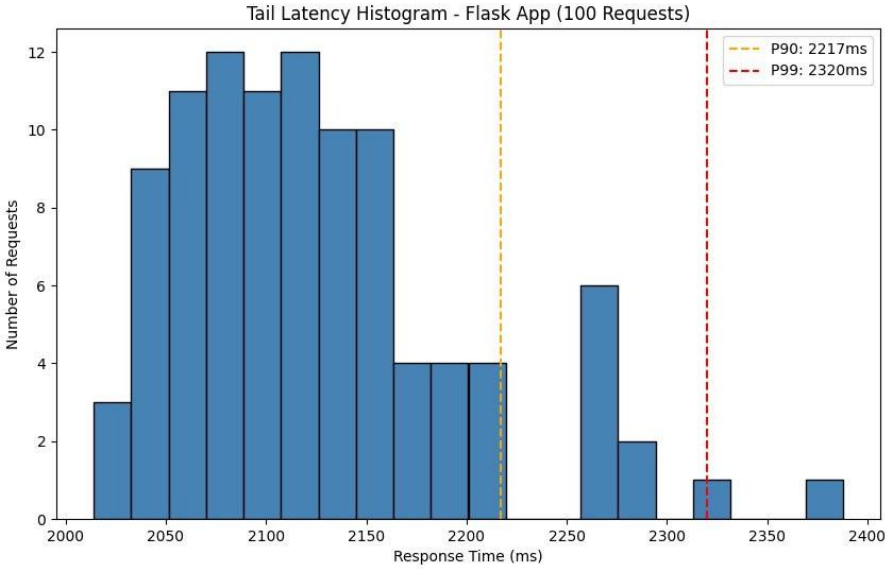


Figure 5: Response time histogram for 100 requests. Orange line = P90 (2217ms), Red line = P99 (2320ms)

Analysis

The histogram shows that most requests (the majority of the distribution) completed between 2050–2150ms. However, a small number of requests experienced significantly longer delays, reaching up to 2387ms. This is the classic tail latency pattern — the exponential distribution of delays means that while most responses are fast, a small percentage are disproportionately slow. In warehouse-scale computing, when hundreds of services are called in parallel, the overall response time is determined by the slowest component, making tail latency a critical concern.

Discussion: VMs vs Containers for Microservices

Which architecture is better for microservices — VMs or containers?

Containers are the better choice for microservices architectures, and the results of this lab clearly demonstrate why.

1. Lightweight and Fast: As shown in Part A, Docker containers run with only 3 processes and minimal memory overhead, compared to a full VM which runs 31+ background processes. This makes containers much faster to start and cheaper to run at scale.

2. Density: Because containers share the host OS kernel, you can run many more containers on the same hardware compared to VMs. This is critical for microservices where you might have dozens of small independent services.

3. Consistency: Containers package the application and its dependencies together, ensuring consistent behavior across development, testing, and production environments.

When VMs are preferred: VMs are better when strong isolation is required (e.g., running untrusted code), when different OS types are needed per workload, or for legacy applications that require a full OS environment.

Discussion Questions

Q: Why does AWS split Nitro into hardware components?

> **A:** AWS splits Nitro into dedicated hardware components (Nitro Cards for networking/storage, Nitro Security Chip, and Nitro Hypervisor) to offload virtualization tasks from the main CPU. This allows EC2 instances to use nearly all CPU and memory for customer workloads rather than hypervisor overhead, improving performance. It also enhances security by isolating the management plane from the data plane.

Q: In which scenarios would you use VMs over containers?

> **A:** VMs are preferred when: (1) strong security isolation is required between tenants, (2) different operating systems are needed per workload, (3) running legacy applications that depend on full OS features, or (4) compliance requirements mandate hardware-level isolation.

Q: How does tail latency change with the number of parallel calls?

> **A:** Tail latency worsens significantly as parallel calls increase. If one request has a 1% chance of hitting a slow response, then with 100 parallel calls, the probability that at least one is slow approaches nearly 100%. The overall response time becomes dominated by the slowest component, which is why tail latency is a major challenge in distributed systems.